

# **SOFTWARE ENGINEERING**

## **UNIT - 3**

### **TOPIC – 3**

#### **CONCEPTUAL MODEL OF UML**

##### **UML (Unified Modeling Language)**

UML, or Unified Modeling Language, is a way to draw diagrams that show how software systems work. It's like drawing a map or blueprint to help people understand what's going on inside a program. Just like how an architect uses blueprints to plan a house, software designers use UML to plan their software before they start building it with code.

##### **CASE Tools for UML**

**CASE tools** (Computer-Aided Software Engineering tools) are special programs that help people draw these UML diagrams. They make it easy to show how the software will work.

Some popular CASE tools are:

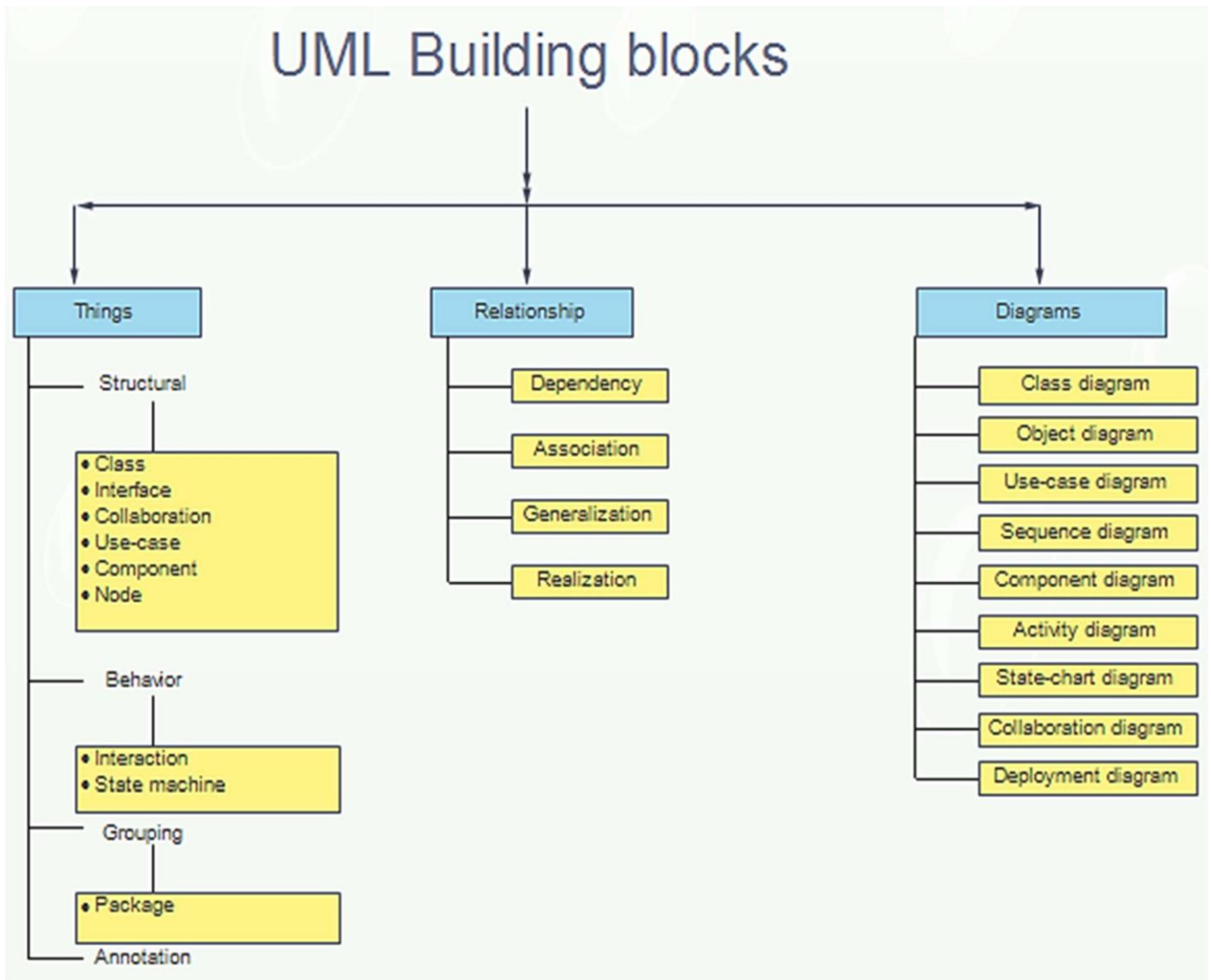
- **Rational Rose** – A well-known tool for drawing UML diagrams.
- **Enterprise Architect** – Helps create and organize diagrams.
- **Microsoft Visio** – A diagramming tool that can also make UML diagrams.
- **Lucidchart** – A web tool for drawing diagrams.
- **StarUML** and **ArgoUML** – Free tools for making UML diagrams.
- **Visual Paradigm** – Offers many features to help design software.
- **Balsamiq**, **Creately**, and **SmartDraw** – Simple tools to make diagrams quickly.
- **Umbrella** – Another tool that helps make UML diagrams.

## UML Building Blocks

UML helps us think about how a software system works by using diagrams.

It focuses on three key points:

1. **Things/Objects:** The parts of the system, like people, data, or tasks.
2. **Relationships:** How these parts connect and work together.
3. **Diagrams:** Pictures that show us how everything fits together.



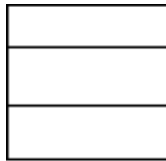
## Parts of UML (Things)

In UML, the "things" are the parts that make up a system. These parts are divided into groups:

1. **Structural Things (Parts That Don't Change)** These describe the solid, unchanging parts of the system, like the structure of a building.

- **Class:** A group of objects with similar features.

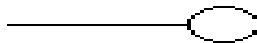
- *Example: A "Car" class has properties like make, model, and color.*



- *Symbol:*      **Class**

- **Interface:** A list of rules or actions that certain objects must follow.

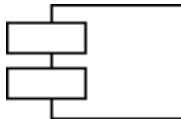
- *Example: A "Drivable" interface makes sure that any class that uses it, like a Car, has a "drive" function.*



- *Symbol:*      **Interface**

- **Component:** A piece of the system that performs a specific job.

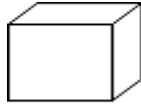
- *Example: A "Payment" component handles all the payments in the system.*



- *Symbol:*      **Component**

- **Node:** A place where the software runs, like a server.

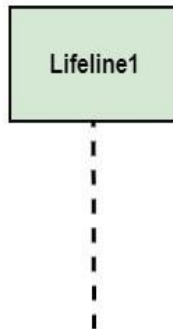
- *Example: A "Database Server" is a node where customer information is stored.*



- Symbol: Node

2. **Behavioral Things (How Things Act or Change)** These describe how the system behaves or changes over time, like how a door can open and close.

- **Interaction:** Shows how different parts of the system talk to each other.
  - *Example: A customer asks for information from the system, and the system sends back the details.*



- Symbol:

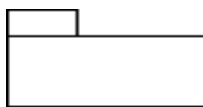
- **State Machine:** Shows the different states something can be in.
  - *Example: A door can be in either "Open" or "Closed" state.*



- Symbol: State

3. **Grouping Things** These organize similar parts of the system into groups.

- **Package:** Groups together related parts.
  - *Example: A "User Management" package might contain parts that deal with users, like login, registration, and permissions.*



- Symbol: Package

4. **Annotational Things** These are notes or comments that give extra information about the system.

- **Annotation:** A comment added to explain something in the diagram.

- *Example: A note that explains why a particular feature is needed.*



- *Symbol:*      **Note**

## **Relationships in UML**

UML diagrams also show how the different parts of a system are connected. These connections are called **relationships**.

Some common types of relationships are:

- **Dependency**: When one thing needs another thing to work.

- *Example: A "Car" class needs an "Engine" class to function properly.*



- *Symbol:*

- **Association**: A simple connection between two things.

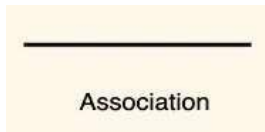
- *Example: A teacher teaches a student, and the student learns from the teacher.*

*They are associated with each other.*

- *Symbol:*



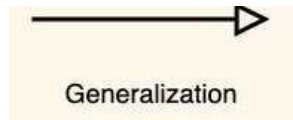
*An "Order" depends on a "Product" to exist because an order is placed for products.*



- A "Doctor" can treat many "Patients," and a "Patient" can see many different "Doctors." Both have a connection (doctor-patient relationship), but they can exist separately.

- **Generalization**: This shows inheritance, where one thing is a more general version of another.

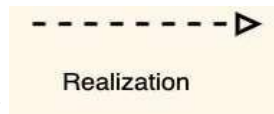
- Example: A "Car" is a general category, and "SUV" and "Sedan" are more specific types of cars.



- Symbol:

- **Realization**: When one thing follows the rules set by another (usually an interface).

- Example: A "Bird" class follows the "Flyable" interface by creating a "fly" function.



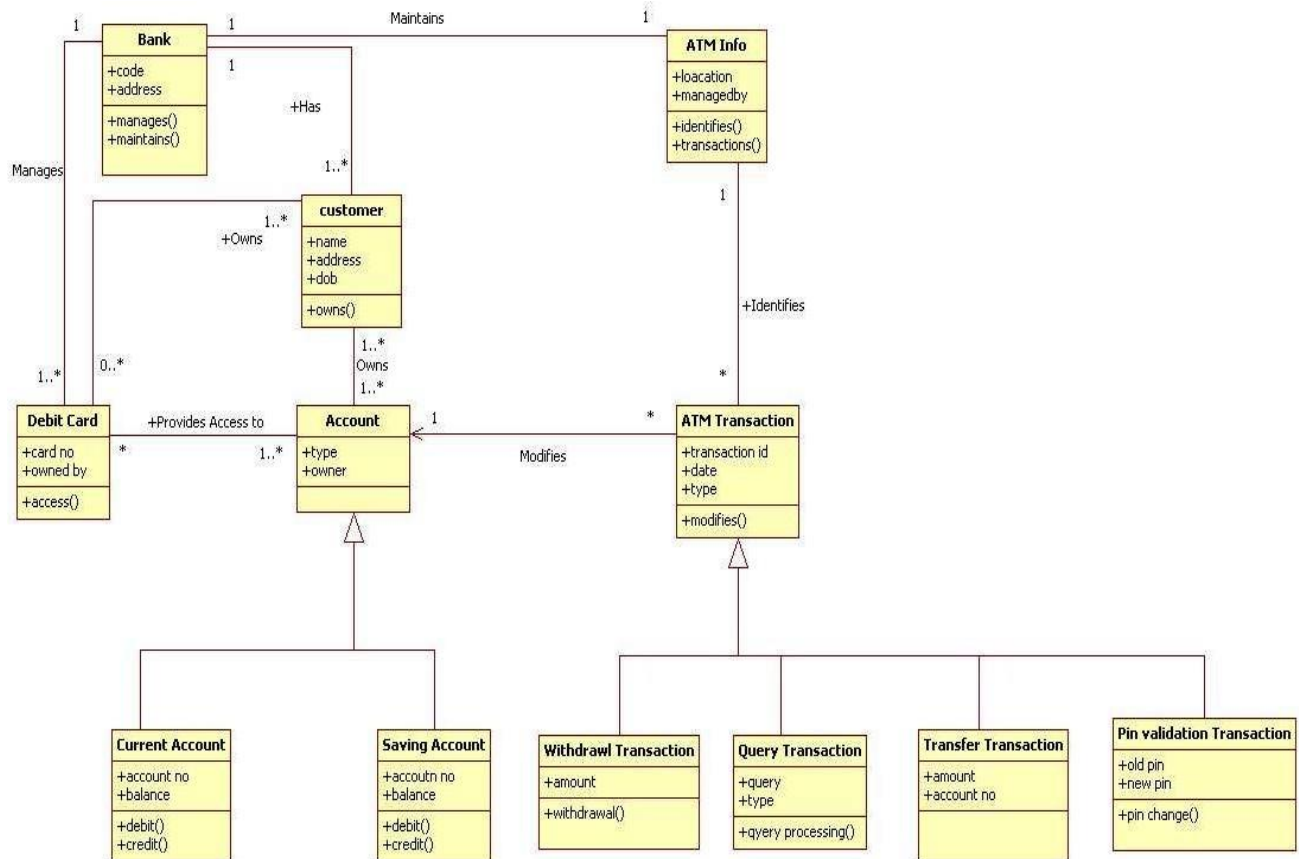
- Symbol:

## Types of UML Diagrams

UML uses different types of diagrams to show various aspects of a system. Each diagram has its own way of explaining how the system works.

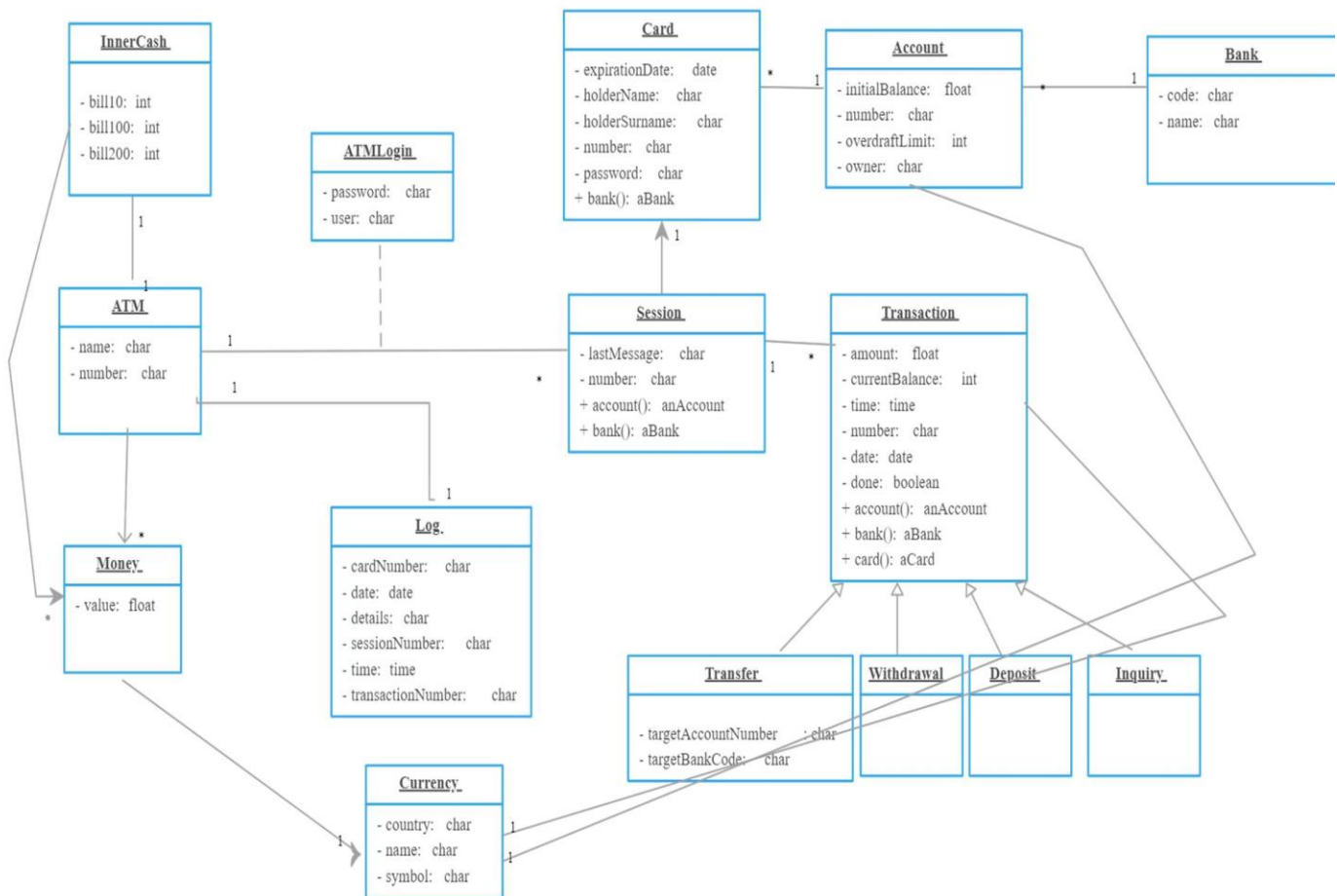
### 1. Class Diagram

- Shows the different classes (types of objects) and how they relate to each other.
- *Example: A "Bank Account" class connected to a "Card" class that handles money transactions.*
- *Example Class Diagram for an ATM system*



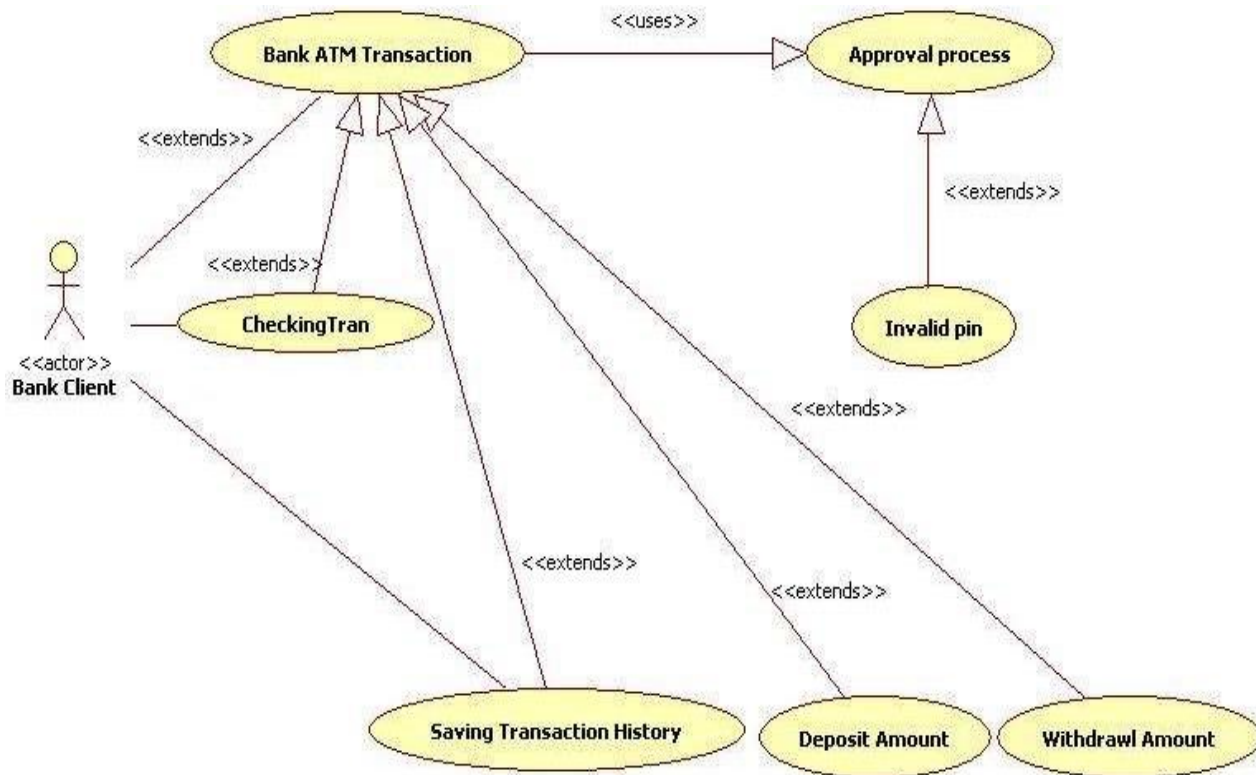
## 2. Object Diagram

- A snapshot of a specific instance in the system at a certain moment.
- Example: A "Debit Card" showing details for one specific user, like card number and expiration date.
- Example Object diagram for the ATM system



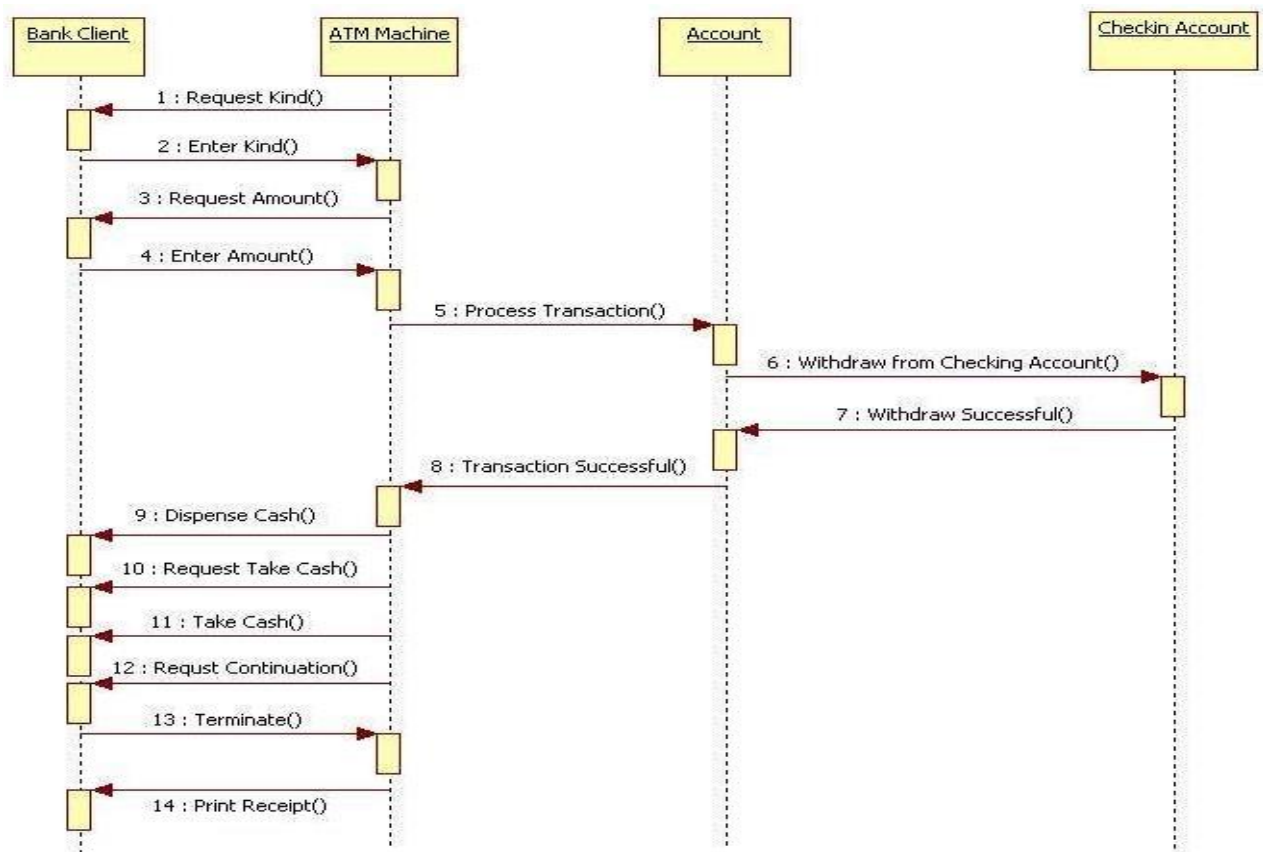
### 3. Use-Case Diagram

- o Describes how users interact with the system.
- o Example: A user withdrawing money from an ATM or checking their balance.
- o *Example Usecase diagram for the ATM system*



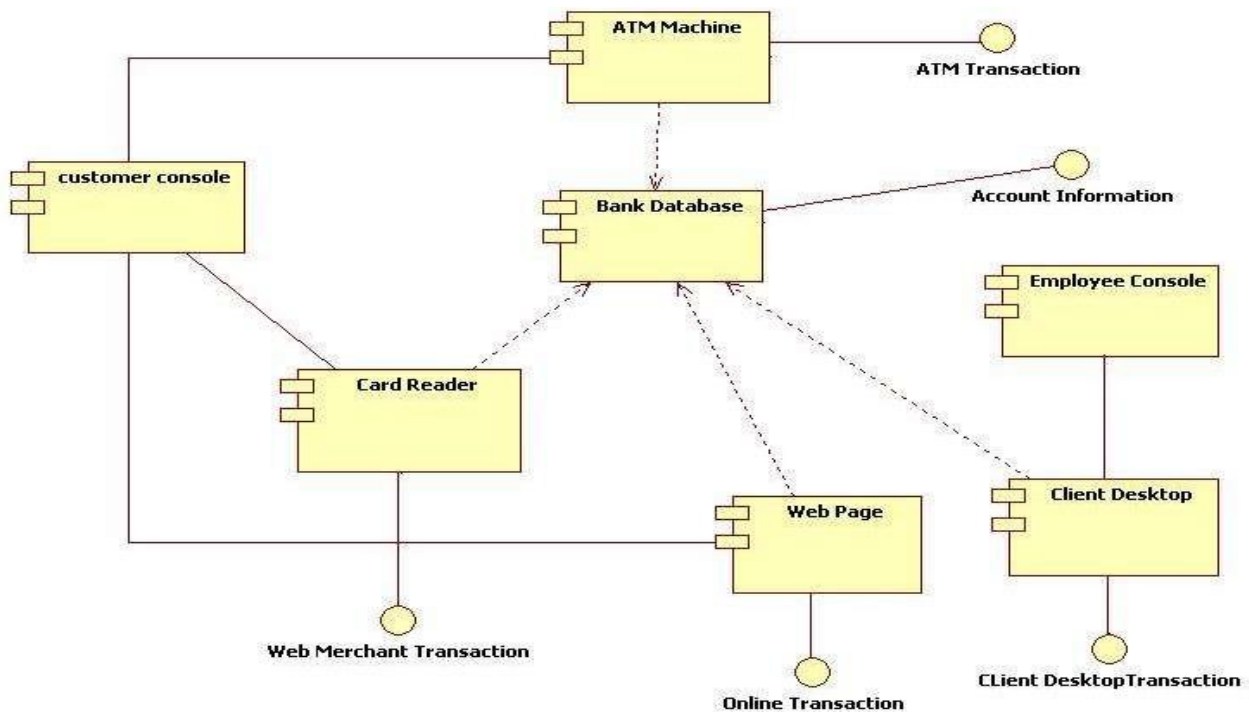
#### 4. Sequence Diagram

- Shows the order in which things happen in a system.
- Example: A user requests cash, the ATM asks the bank, and money is dispensed.
- *Example Sequence diagram for the ATM system*



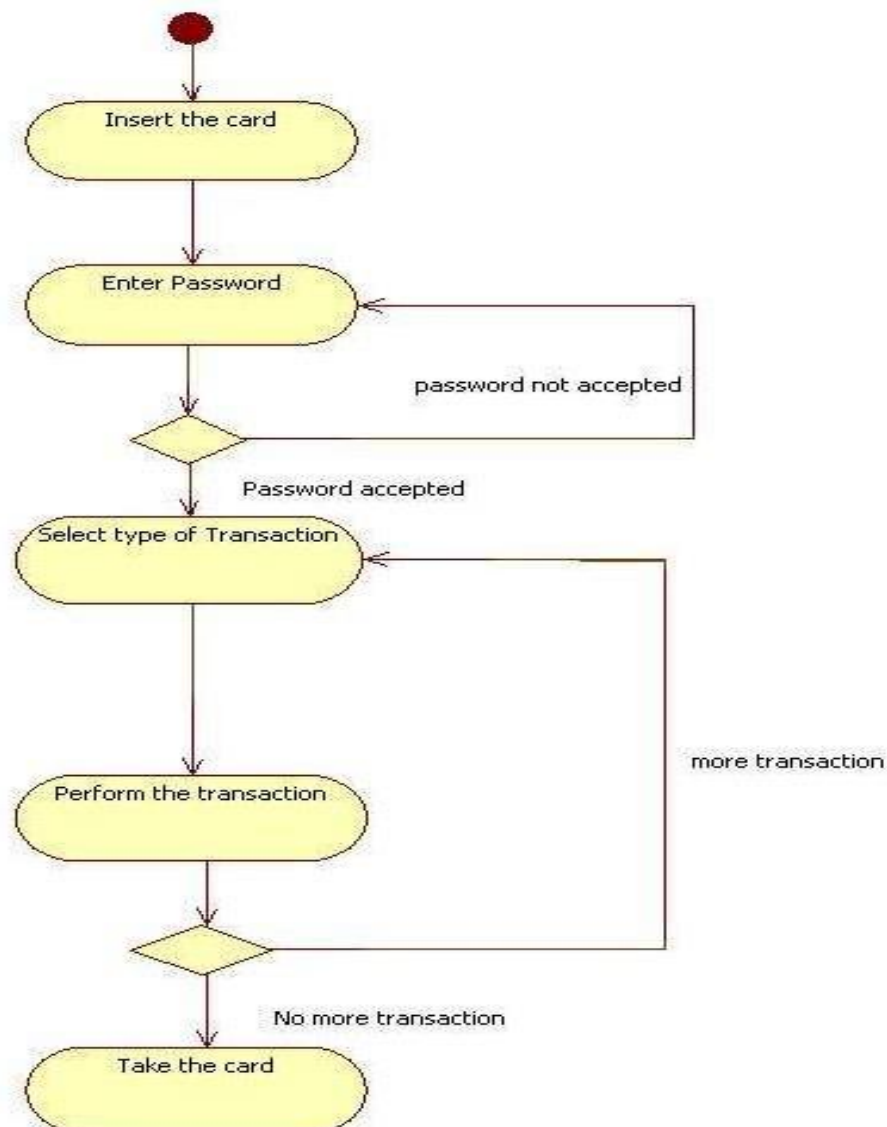
## 5. Component Diagram

- Shows how the different parts (components) of the system fit together.
- Example: An ATM machine's software connects to the bank's database to process a transaction.
- *Example Component diagram for the ATM system*



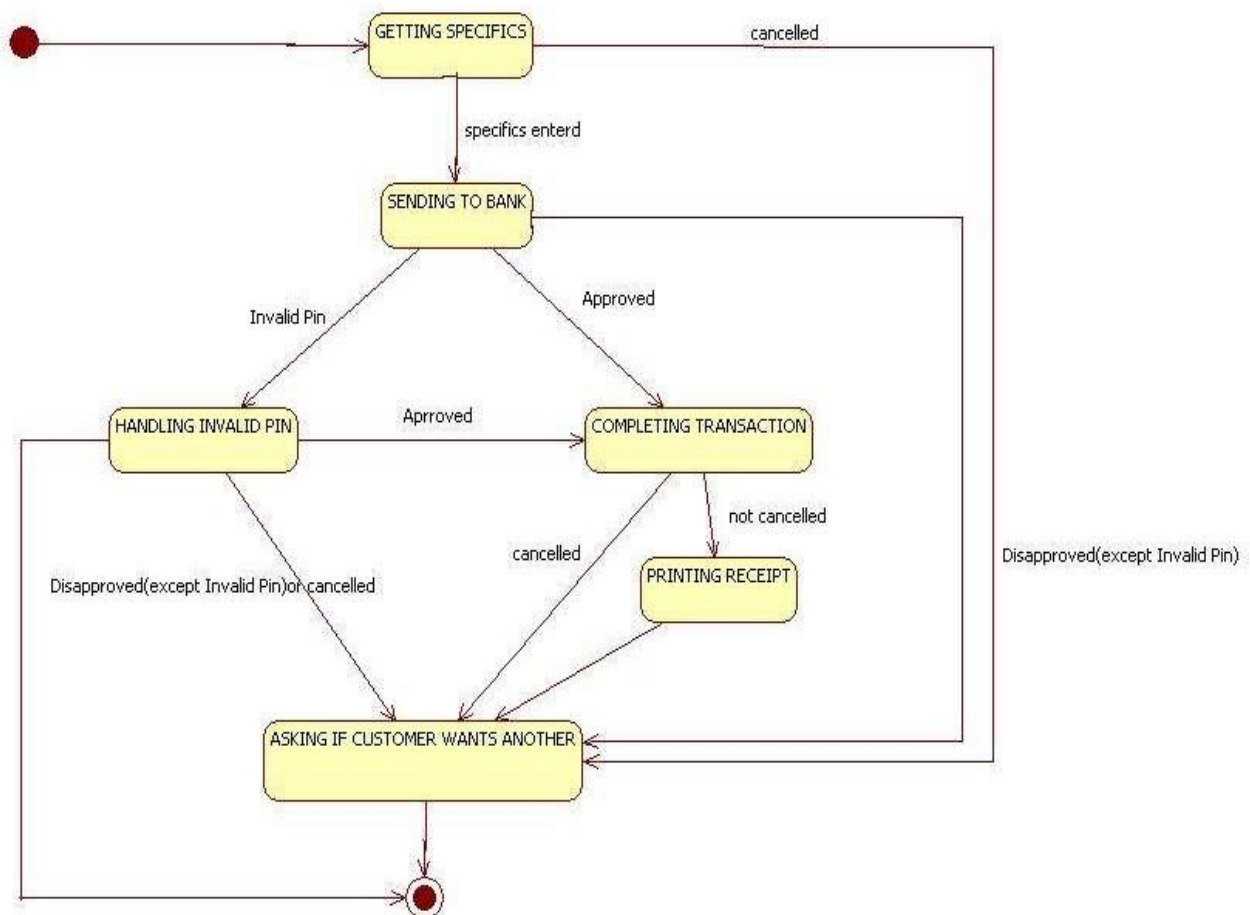
## 6. Activity Diagram

- Shows the steps in a process.
- Example: The process of withdrawing money from an ATM: insert card, enter PIN, select amount, and get cash.
- *Example Activity diagram for the ATM system*



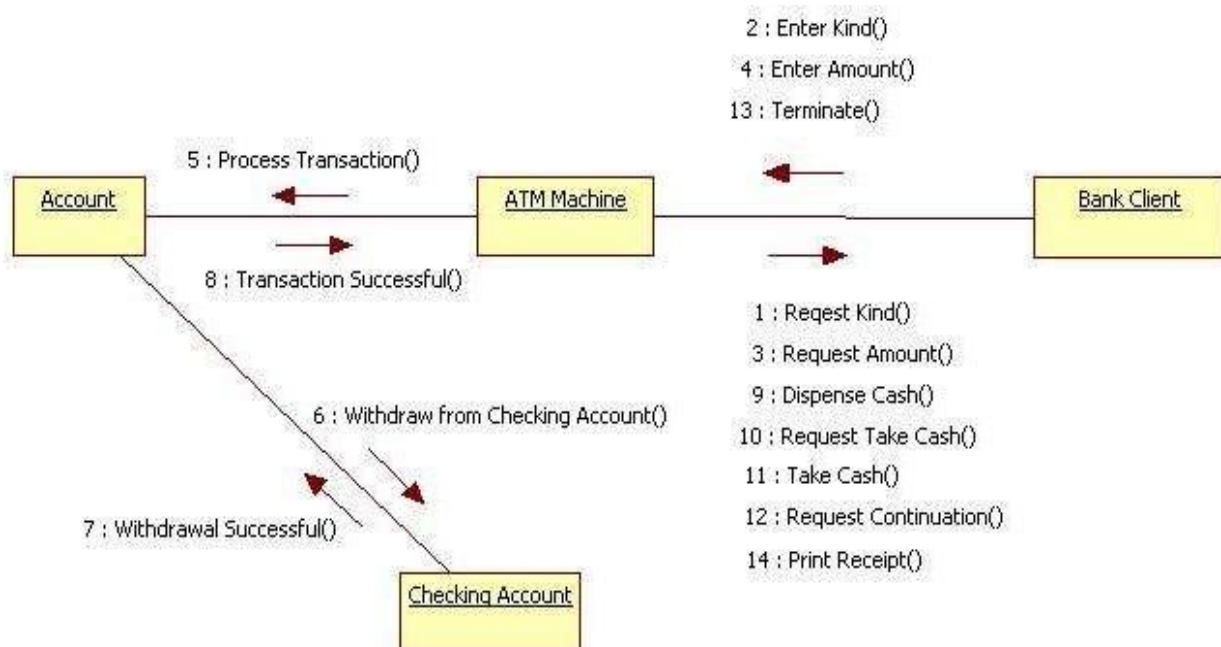
## 7. State-Chart Diagram

- o Describes the different states an object can be in and how it changes between them.
- o Example: An ATM machine can be "Idle" when not in use and "Dispensing Cash" when a user withdraws money.
- o *Example State-chart diagram for the ATM system*



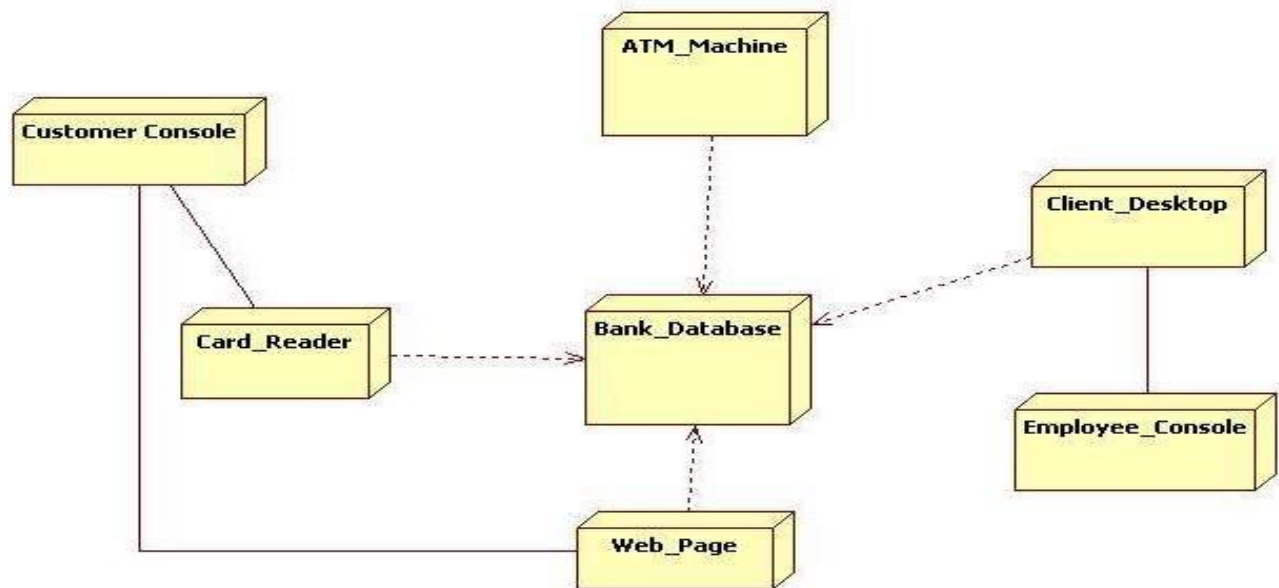
## 8. Collaboration Diagram

- o Focuses on how different parts of the system work together to achieve something.
- o Example: How the ATM, the user, and the bank's server work together during a cash withdrawal.
- o *Example Collaboration diagram for the ATM system*



## 9. Deployment Diagram

- Shows where the physical parts of the system are located and how they are connected.
- Example: An ATM machine at a bank connects to the bank's server, which might be in a different city.
- *Example Deployment diagram for the ATM system*



UML makes it easier for people working on software to understand how everything fits together. By using diagrams to show how the system will work, teams can plan, communicate, and organize their ideas before they start building the actual software. UML is a simple but powerful way to visually map out a complex system, making it much clearer for everyone involved.

# SOFTWARE ENGINEERING

## UNIT – 3

## TOPIC – 4

### UML DIAGRAMS – USECASE

#### Use Case Diagrams

A Use Case Diagram is a type of diagram that helps us understand how a software system works from the user's point of view. It shows the people (called actors) or other systems that interact with the software, and the specific actions they can perform. This diagram makes it easy to see what the system can do and how it works.

#### Purpose of a Use Case Diagram

The main reason for using a Use Case Diagram is to show the functions of a system in a simple way. This diagram is very useful when you need to understand the overall picture of what the system does, without diving into too many details. It does three main things:

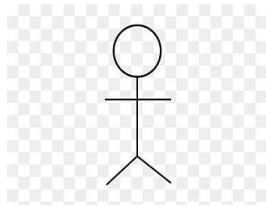
1. **Shows the Actors:** It identifies who is using the system (like a customer or another system).
2. **Describes the Actions:** It explains what actions or tasks the user can do with the system (like withdrawing money from an ATM).
3. **Defines the System's Boundaries:** It clearly shows what is **inside the system** (what the system does) and what is **outside the system** (the users or other systems).

#### Key Parts of a Use Case Diagram

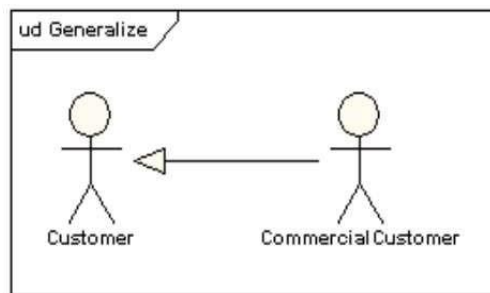
There are several important parts (symbols) in a Use Case Diagram. Each one plays a specific role in explaining how the system works.

**1. Actor:** An actor is anyone or anything that interacts with the system. This could be a person, another software system, or even a piece of hardware.

- **Example:** In an ATM system, the actor could be the "Customer" using the ATM or the "Bank" that processes the transactions.
- **Symbol:** It is represented by a **stick figure**. This makes it easy to recognize as someone or something interacting with the system.



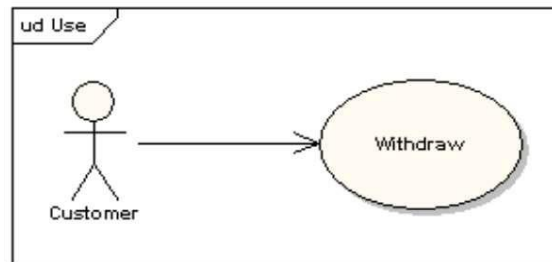
- **Generalization of Actors:** Actors can have a hierarchical relationship. One actor can "generalize" another, meaning one type of actor shares similarities with another but may also have additional roles.



- In the image, we see a Customer Actor and a Commercial Customer actor. A Commercial Customer is a type of customer, but with more specific responsibilities. The arrow between them shows this generalization, meaning all commercial customers are customers, but not all customers are commercial customers.

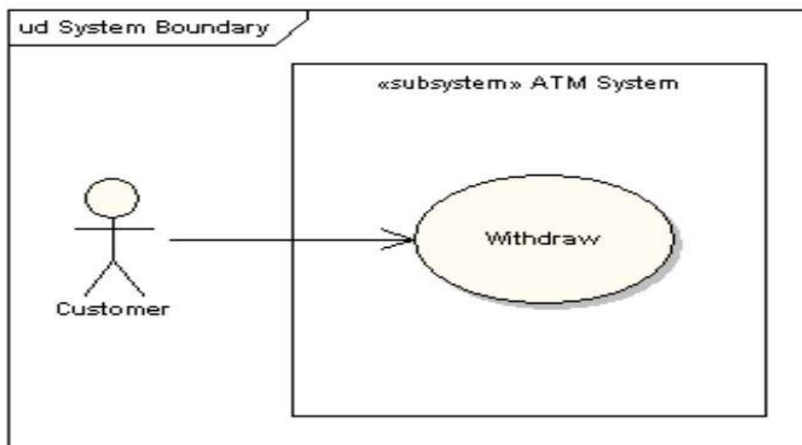
## 2. Use Case

- **Definition:** A use case is an action or task that the actor can do with the system. It is like a function or activity the system provides to the user.
- **Example:** For an ATM system, use cases could include tasks like "Withdraw Cash," "Check Balance," or "Transfer Funds."
- **Symbol:** A use case is shown as an **oval** with the name of the action written inside.

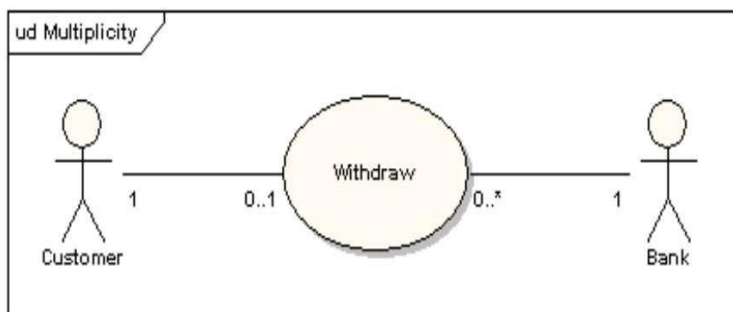


### 3. System Boundary

- **Definition:** The system boundary is a box or rectangle that surrounds the use cases. It helps to show what is part of the system and what is not. Everything inside the boundary is controlled by the system, and everything outside interacts with it.
- **Symbol:** The boundary is represented by a **rectangle** that encloses all the use cases.

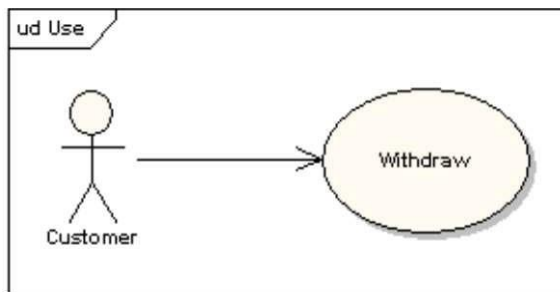


**4. Associations (Lines):** An Association is a straight line that shows a connection between an actor (a person or system) and a use case (a task or action). It means the actor is involved in the task, but it doesn't say who starts the action.



*Example:* A line between a Customer and a Login task means the customer is part of the login process, but it doesn't tell us who begins the login.

**Directed Association:** A Directed Association is like an association, but it has an arrow. The arrow shows who starts or controls the action. It points to the task, meaning the actor is the one who starts the action.



*Example:* If there's an arrow from a Customer to a Withdraw task, it means the customer starts the process of withdrawing money. Without the arrow, it just shows involvement, but we don't know who initiates the task.

**Multiplicity:** Multiplicity tells us how many times an actor can be involved in a task. Numbers next to the line show this.

*Example:* If a Customer is linked to a Purchase task with multiplicity 0..1, it means they can either make one purchase or none at all. If it's 1..\*, it means the customer can make as many purchases as they want.

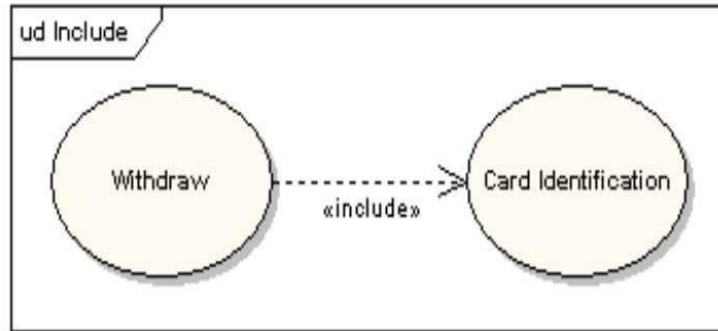
Use multiplicity when you need to explain how many times a person or system can do something. For example, a customer might log in only once, but they might make many purchases.

## **Special Relationships in a Use Case Diagram**

Sometimes, use cases can be related to each other in special ways. These relationships help make the diagram clearer.

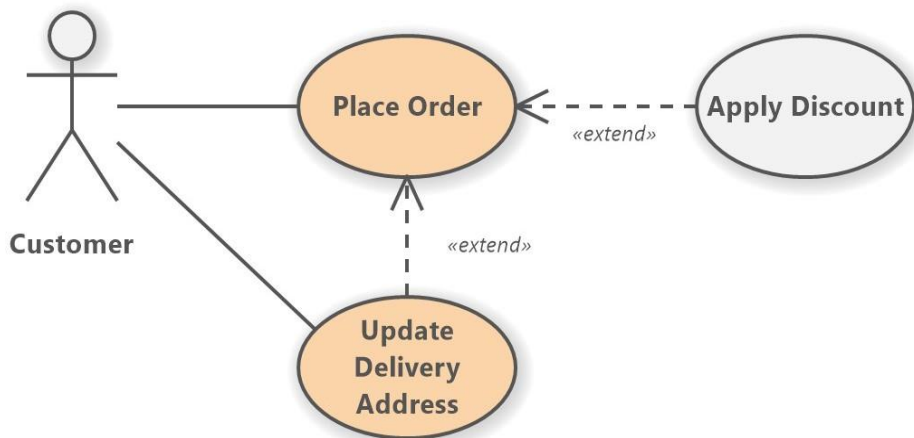
**1. Include:** This means that one use case **must always include** another use case to complete its task. It is a way of saying that a certain action is part of a larger action.

- **Example:** In an ATM system, before a customer can "Withdraw Cash," they must first "Authenticate" themselves (by entering a PIN). So, "Authenticate User" is always included in "Withdraw Cash."
- **Symbol:** This relationship is shown with a **dashed arrow** labelled <<include>>.



**2. Extend:** This means that one use case can **add extra steps** or **features** to another use case, but only if needed. It is like an optional action that only happens under certain conditions.

- **Example:** After withdrawing cash, the customer might want to check their balance. This means the "Check Balance" use case extends the "Withdraw Cash" use case.
- **Symbol:** This is shown with a **dashed arrow** labelled <<extend>>.



## Steps to Create a Use Case Diagram for an ATM System

Use Case Diagram for an ATM system.

### Step 1: Identify the Actors

- **Customer:** This is the person using the ATM to do tasks like withdrawing money or checking their account balance.
- **Bank:** This represents the system or organization that processes the customer's transactions.

### Step 2: Identify the Use Cases

- **Withdraw Cash:** The customer takes money out of their account using the ATM.
- **Check Balance:** The customer checks how much money is in their account.
- **Deposit Cash:** The customer puts money into their account.
- **Transfer Funds:** The customer moves money from one account to another.
- **Change PIN:** The customer changes the PIN number used to access the ATM.

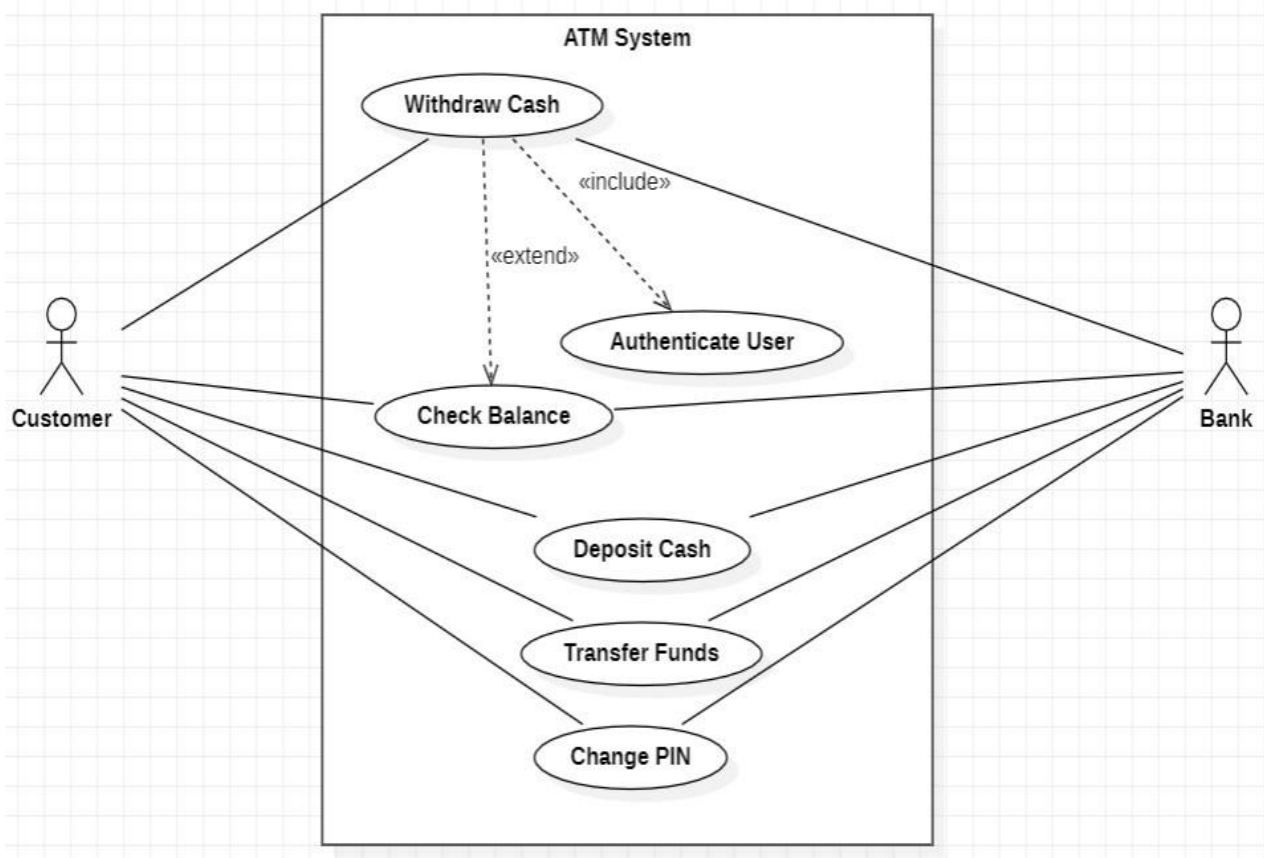
### Step 3: Draw the Diagram

- First, draw a large rectangle to represent the system boundary and label it "ATM System."
- Inside the rectangle, draw ovals for each use case (Withdraw Cash, Check Balance, etc.).
- Outside the rectangle, draw stick figures for the actors (Customer, Bank).
- Connect the actors to the use cases using straight lines to show what actions the actors can perform.

### Step 4: Use Include and Extend

- **Include:** The "Withdraw Cash" use case always includes "Authenticate User," since the customer must be authenticated before they can withdraw money. Draw a dashed arrow from "Withdraw Cash" to "Authenticate User" and label it <<include>>.

- **Extend:** The "Check Balance" use case extends "Withdraw Cash" because, after withdrawing cash, the customer may want to check their balance. Draw a dashed arrow from "Check Balance" to "Withdraw Cash" and label it <<extend>>.



# SOFTWARE ENGINEERING

## UNIT – 3

## TOPIC – 5

### UML DIAGRAMS – CLASS

#### Class Diagram

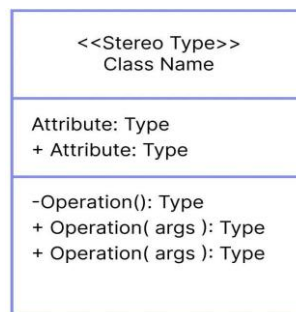
A Class Diagram is a key building block in object-oriented modeling. It represents the static structure of a system by describing:

- **Classes:** The blueprint for objects.
- **Attributes:** Characteristics of the classes.
- **Methods:** The behaviors or actions associated with the class.
- **Relationships:** How the classes interact with each other.

Class diagrams are crucial in software development because they help engineers and stakeholders understand how the different components of a system fit together.

#### Components of a Class Diagram:

1. **Classes:**
  - o A class in UML is a blueprint for objects. It is depicted as a rectangle divided into three sections:
    - **Top section:** Contains the name of the class.
    - **Middle section:** Lists the attributes (properties or data) of the class.
    - **Bottom section:** Shows the methods (functions or behaviors) that the class can perform.



## 2. Attributes:

- o Attributes are variables that represent the state or properties of a class.
- o They describe what an object of the class knows or possesses.
- o Examples:
  - A Car class might have attributes like color, model, and speed.
  - A Person class might have attributes like name, age, and height.
- o Attributes can be:
  - **Private (-):** Only accessible within the class.
  - **Public (+):** Accessible from outside the class.
  - **Protected (#):** Accessible by the class and its subclasses.

## 3. Methods:

- o Methods represent the functions or behaviors that objects of the class can perform.
- o They describe what actions an object of the class can take.
- o Examples:
  - A Car class might have methods like drive() and stop().
  - A Person class might have methods like walk(), talk(), and sleep().

## 4. Relationships:

- o The relationships between classes define how different classes are connected and interact with each other. UML supports various types of relationships, including:

### **Association:**

- Represents a general connection between two classes. It implies that objects of one class interact with objects of another class.
- *Example: A Teacher and Student. The association shows that teachers teach students, and students are taught by teachers.*
- Shown by a solid line connecting the two classes.



### **Inheritance (Generalization):**

- Also known as Generalization, this relationship indicates that one class inherits attributes and methods from another class.
- *Example: Dog is a subclass of Animal, meaning that all dogs inherit properties like legs and eyes from the Animal class, but dogs also have additional properties like barking.*
- Shown by a solid line with a hollow arrow pointing toward the parent class.



### **5. Aggregation:**

- Represents a whole-part relationship where the part can exist independently of the whole.
- *Example: A classroom can exist without its students. Even if a student leaves, the classroom still exists.*
- Shown by a solid line with a hollow diamond near the class that represents the whole.

### **6. Composition:**

- A stronger form of aggregation where the part cannot exist independently of the whole.
- *Example: A house and its rooms. If the house is destroyed, the rooms no longer exist.*
- Shown by a solid line with a filled diamond near the class that represents the whole.

### **7. Dependency:**

- A weaker relationship where one class depends on another to perform its function.
- *Example: A Printer depends on a PrintJob. The printer class depends on the presence of a print job to function.*
- Shown by a dashed arrow from the dependent class to the class it depends on.

### **8. Realization:**

- Represents that a class implements an interface or a contract.
- *Example: If a class Car implements an interface Vehicle, it must define all methods declared in the Vehicle interface, such as start() and stop().*
- Shown by a dashed line with a hollow arrow pointing towards the interface.

## Drawing a Class Diagram for an ATM System

To illustrate how class diagrams are used, consider an ATM system. When modeling such a system, you break it down into classes, define their attributes and methods, and specify how they interact with each other.

### **Steps to Create an ATM Class Diagram:**

#### 1. Identify the Main Classes:

- o Think about the key components of the ATM system. These will form the classes.
- o *Examples of classes:*
  - **ATM:** The machine itself.
  - **Customer:** The person using the ATM.
  - **Bank Account:** The account that holds the customer's money.
  - **Transaction:** Actions like withdrawal, deposit, etc.

#### 2. Define Attributes for Each Class:

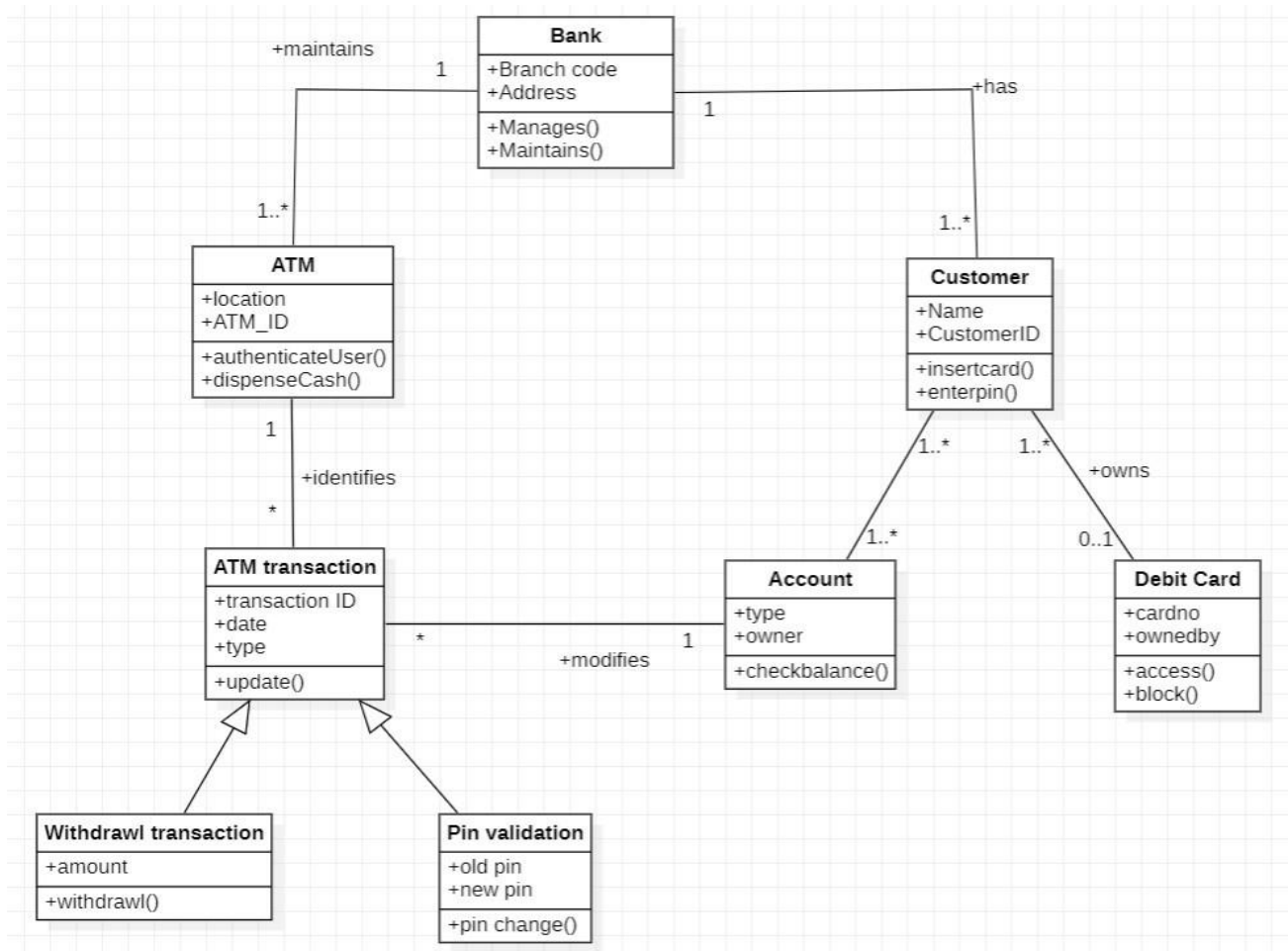
- o List the important properties each class should have.
- o *Examples:*
  - **ATM:** location, ATM\_ID.
  - **Customer:** name, customerID.
  - **Bank Account:** accountNumber, balance.

#### 3. Define Methods for Each Class:

- o Methods are the actions that each class can perform.
- o *Examples:*
  - **ATM:**
    - authenticateUser(): Verify if the user is valid.
    - dispenseCash(): Give cash to the customer.
  - **Customer:**
    - insertCard(): The customer inserts their card into the ATM.
    - enterPIN(): The customer enters their PIN to authenticate.
  - **Bank Account:**
    - checkBalance(): Check the current balance in the account.
    - withdrawMoney(): Withdraw a specified amount of money.

4. Show the Relationships Between the Classes:

- o **Association:** There is a relationship between the ATM and the customer, as well as between the ATM and the bank account.
- o **Dependency:** The ATM depends on a Transaction to perform operations like withdrawal.
- o **Composition:** An ATM contains components like a Card Reader, Cash Dispenser, and Display, and these components cannot exist without the ATM.



## **Examples of Class Diagrams in the Real World**

### **1. E-Commerce System:**

- o Classes:
  - Customer: Represents a user of the e-commerce platform.
  - Product: Represents an item that is available for purchase.
  - Order: Represents a customer's order, which includes multiple products.
- o Relationships:
  - Association: A customer can place multiple orders.
  - Composition: An order consists of several products.

### **2. Library System:**

- o Classes:
  - Book: Represents a book in the library.
  - Member: Represents a library member.
  - Librarian: Represents the person managing the library.
- o Relationships:
  - Association: A member borrows books.
  - Inheritance: A librarian and a member are both people, so they inherit from a Person class.

# **SOFTWARE ENGINEERING LAB**

## **UNIT – 3**

## **TOPIC – 6**

### **UML DIAGRAMS – SEQUENCE**

#### **Sequence Diagram**

- A sequence diagram is a type of UML diagram that illustrates how objects in a system interact with each other over time.
- It shows the order in which messages are sent between objects, helping visualize the sequence of interactions in a system.
- This is particularly useful when trying to understand the flow of actions or events between different components of a system.

#### **2. Components of a Sequence Diagram**

##### **1. Actors:**

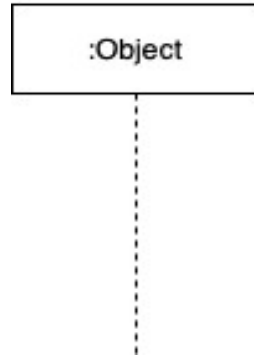
- Actors represent external entities interacting with the system.
- These could be people (like users) or other systems (like a database or another software system).
- In a sequence diagram for an ATM system, for example, the Customer would be an actor.



##### **2. Objects:**

- Objects are parts of the system that perform specific actions.
- Each object represents a class or an entity that plays a role in the interaction.

- For example, in an ATM system, objects could include:
  - The ATM machine itself.
  - The Bank Database that stores the customer's account information.



### 3. **Lifelines:**

- A Lifeline is a dotted vertical line that extends below each object, representing the object's existence over time.
- The activation box on the lifeline shows when the object is actively performing an action (e.g., processing a transaction).
- For example, if a customer initiates a withdrawal, the ATM machine's lifeline shows the duration during which it processes the request.



### 4. **Messages:**

- Messages are arrows between objects indicating the communication between them.
- Each arrow represents a specific action or message being passed from one object to another.



- o The arrow shows:
  - Direction of the message (who sends and who receives it).
  - Order of the messages.
- o In the ATM system example:
  - The Customer might send a message to the ATM: insertCard().
  - The ATM then sends a message to the Bank Database: authenticateUser().

### **3. Example: Sequence Diagram for an ATM System**

Let's break down how a sequence diagram might look for an ATM system, considering the interactions for a cash withdrawal process.

#### **Actors:**

- Customer (the person using the ATM).

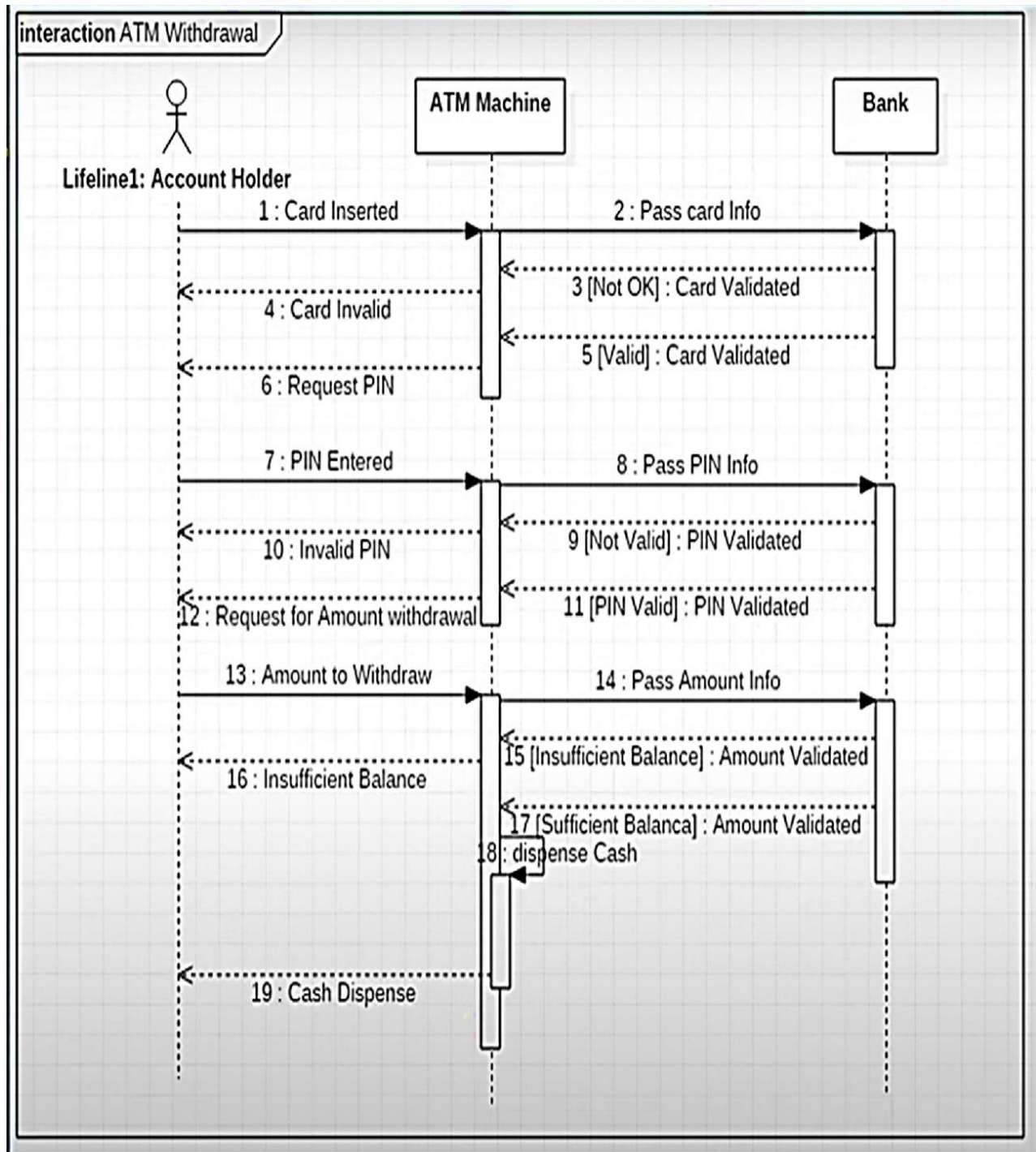
#### **Objects:**

- ATM (the machine the customer is interacting with).
- Bank Database (the system that holds the customer's account details).

#### **Messages:**

1. The Customer inserts the card into the ATM: insertCard().
2. The ATM requests the PIN from the customer: requestPIN().
3. The Customer enters the PIN: enterPIN().
4. The ATM sends the entered PIN to the Bank Database for verification: authenticateUser().
5. The Bank Database verifies the PIN and sends a response: authenticationResult().
6. If the authentication is successful, the ATM prompts the Customer to enter the withdrawal amount: requestAmount().
7. The Customer enters the withdrawal amount: enterAmount().

8. The ATM sends a request to the Bank Database to withdraw money: `withdrawMoney()`.
9. The Bank Database confirms the transaction and updates the balance: `transactionConfirmation()`.
10. The ATM dispenses the cash and ends the transaction: `dispenseCash()`.



## **Purpose of Sequence Diagrams**

- Sequence diagrams are used to model the dynamic behavior of a system. They help in understanding:
  - How different parts of the system work together to complete a process.
  - The order in which actions or events happen in the system.
  - The flow of messages between different objects over time.

In an ATM system, for example, the sequence diagram helps visualize the step-by-step interaction between the customer, the ATM machine, and the bank's database during a transaction like withdrawing cash. It highlights how the ATM system interacts with the external user and bank database to complete the transaction process smoothly.

# SOFTWARE ENGINEERING

## UNIT – 3

### TOPIC – 7

## UML DIAGRAMS – COMPONENT

### Component Diagram

A **Component Diagram** is a type of UML (Unified Modeling Language) diagram that shows how a system is organized into **smaller parts** called **components**. These components work together to form the **whole system**. It helps to understand:

- **What parts** the system is made of.
- **How these parts connect** to each other.
- **How the system works** by looking at how each component functions.

### What are Components?

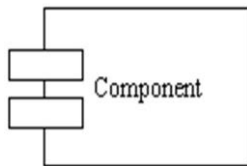
- **Components** are the **building blocks** of a system, just like parts of a machine.
- Each component is a **self-contained unit** that does a specific job.
- Components **communicate** with each other through **interfaces**, which are defined connection points that allow components to work together.
- **Example:**
- *In a car, components include:*
- **Engine:** Provides power.
- **Brakes:** Stops the car.
- **Transmission:** Controls how the power is used to move the car.
- Similarly, in a software system, components could be:
- **Login system.**
- **Database.**
- **User interface.**

## Characteristics of Components

- **Reusable:** Components can often be used in different parts of a system or even in **other systems**. This reduces duplication and saves development time.
- **Replaceable:** If a component stops working or needs to be updated, it can be replaced without affecting the other parts of the system.
- **Modular:** Components are designed to work **independently**, making the system easier to maintain and update.

## Symbols Used in a Component Diagram

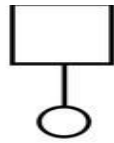
- **Component:**
- Represents a **part** of the system, like a software function or hardware part.
- **Symbol:** A rectangle with two smaller rectangles (tabs) sticking out from the left side.



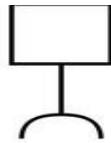
- Example: A **login system** or a **database**.
- **Interface:**
- Shows **how a component interacts** with other parts of the system, like a connection point where other components can communicate.
- **Symbol:** A small circle (called a "lollipop") connected to the component.



- **Provided Interface:**
- This means the component **offers a way** for others to use its functionality.
- **Symbol:** A lollipop symbol (circle) connected to the component.
- **Required Interface:**
- This means the component **needs** something from another component to work properly.
- **Symbol:** A half-circle (socket) connected to the component.



**Provided  
Interface**



**Required  
Interface**

- 
- **Dependency:**
- Shows that one component **depends** on another to function properly.
- **Symbol:** A dashed arrow pointing from the dependent component to the component it relies on.



- **Connector:**
- Shows a **connection** or communication between two components.
- **Symbol:** A solid line with an arrow between components.



### **Example: Key Components of an ATM System**

The ATM is made up of several components, and each one has its own function. Here's how they work together:

1. **ATM Machine (Main Component):**
  - o This is the physical machine you see and interact with.
  - o It has several important sub-components:

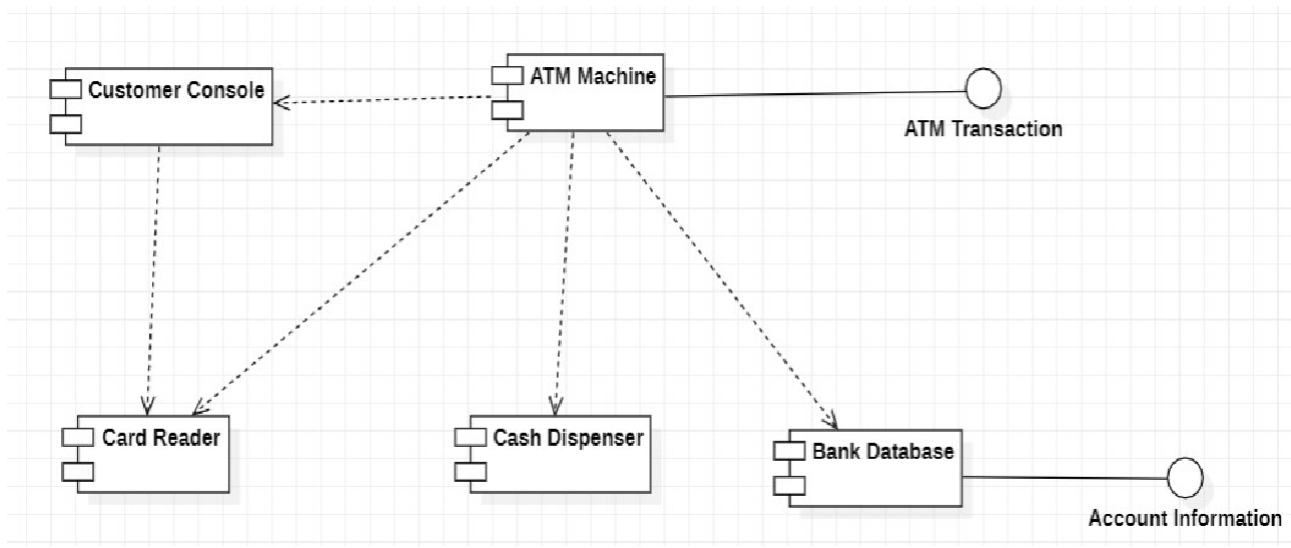
1. User Interface: The screen where options like “Withdraw” or “Check Balance” are shown, and you interact with the machine.
2. Card Reader: The slot where you insert your card. It reads the card's information, like your account details.
3. Cash Dispenser: The part that gives you money after a successful withdrawal.

**2. Bank Database (Main Component):**

- o This is the server that stores your account information, such as your balance and transaction history.
- o It checks if you have enough money for a withdrawal and updates your account after the transaction.

**3. Network (Main Component):**

- o This is the communication line between the ATM and the bank’s database.
- o It transfers your requests (like a withdrawal) to the bank’s system and sends back confirmation (whether the transaction was successful or not).



**How Components Work Together in the ATM System**

**1. Customer Interaction:**

- o The customer interacts with the ATM machine by using the User Interface and Card Reader.

**2. Communication:**

- o The ATM communicates with the Bank Database through the Network to confirm account information and transaction status.

**3. Actions:**

- o If the transaction is approved, the Cash Dispenser will give the customer their money.

**Why Component Diagrams Are Useful**

- Helps in understanding how a complex system is organized by showing the individual parts and how they work together.
- Makes systems easier to manage since each component can be changed, replaced, or updated without affecting the whole system.
- Ensures modularity: Systems are broken down into smaller, manageable pieces, making them easier to develop, maintain, and scale.

# BUILD TOOLS: INTRODUCTION TO MAVEN

## **1. Understanding Maven**

**Definition:** Maven is a tool used for managing and building software projects, mainly in Java. It automates key tasks such as compiling code, running tests, and packaging the final software. Maven also manages dependencies by automatically downloading and organizing external libraries that your project needs.

In simple terms, Maven helps developers by simplifying repetitive tasks and keeping projects well-organized, ensuring that all the necessary tools and libraries are in place.

## **2. The Importance of Maven in Software Development**

- **Without Maven:** Developers must handle every step manually, including finding libraries, compiling code, running tests, and packaging the project.
- **With Maven:** Maven automates these tasks. It structures your project, downloads the libraries, compiles the code, runs tests, and packages everything for you. This results in faster development and fewer errors.

**Example:** If you're building a Java project that requires a library for image processing, you normally need to search for the correct library, download it, and set it up manually. **With Maven,** you just specify the library in the POM file, and Maven automatically fetches and integrates it into your project.

## **3. Maven Operations**

Maven relies on a **POM file** (Project Object Model) to function. This file serves as an instruction guide for Maven, telling it what libraries are needed and how to build the project.

## **4. Exploring the POM File (Project Object Model)**

The **POM file** is the backbone of any Maven project. It is written in XML and specifies how the project is structured, what dependencies are needed, and how the project should be built.

**Simple Example of a POM File:** Think of the POM file like a recipe. It lists the ingredients(libraries) and the steps needed to create the final product.

```
xml
Copy code
<project>
  <groupId>com.example</groupId>
  <artifactId>my-project</artifactId>
  <version>1.0</version>

  <dependencies>
    <dependency>
      <groupId>org.example</groupId>
      <artifactId>image-library</artifactId>
      <version>2.1</version>
    </dependency>
  </dependencies>
</project>
```

Breaking it down:

- **groupId:** Identifies the organization or group that makes the project.
- **artifactId:** The name of your project (e.g., my-project).
- **version:** The version number of your project (e.g., 1.0).
- **dependencies:** This section lists the libraries your project needs. Here, it specifies the **image-library** from org.example.
- **Without Maven:** You would need to find and download the **image-library** manually.
- **With Maven:** You just list it in the POM file, and Maven automatically downloads it for you.

## **5. Build Lifecycle in Maven**

Maven follows a series of steps called the **build lifecycle** to create and manage your

project.

- **Without Maven:** You have to manually compile your code, run tests, and package everything into a JAR file.
- **With Maven:** You can run a command like `mvn package`, and it will automatically do all these steps in the right order:
  1. **Clean:** Deletes old files from previous builds.
    - Command: `mvn clean`
  2. **Test:** Runs tests to check if the code works.
    - Command: `mvn test`
  3. **Package:** Bundles everything into a single file for you to use or share.
    - Command: `mvn package`
  4. **Install:** Installs the package into your local repository.
    - Command: `mvn install`
  5. **Deploy:** Sends the package to a remote repository for others to use.
    - Command: `mvn deploy`
- **Example:** You can use commands like `mvn clean`, `mvn compile`, or `mvn package` to perform specific actions during the build process.

## **6. Understanding JAR, WAR, and Target Files**

In the **package** phase, Maven produces files that vary depending on project type:

- **JAR (Java ARchive):** Contains compiled Java code, resources, and dependencies for standalone applications.
- **WAR (Web Application Archive):** Packages web components (servlets, JSP, HTML, CSS) for web applications. Deploying this on a server enables web access.

All these files are stored in the **target folder**, created automatically during the build and containing both intermediate and final build files.

## **7. Managing Libraries with Maven Repositories**

Maven uses **repositories** to store and find the libraries your project needs.

## Types of Repositories:

1. **Local Repository:** This is on your computer. Maven checks here first to see if you have the libraries you need.
2. **Central Repository:** This is an online store of libraries managed by the Maven community. If a library isn't in your local repository, Maven downloads it from here.
3. **Remote Repository:** If a library is not available in the central repository, you can specify an additional location for Maven to look.

**Example:** When your project needs a library, Maven will first look in your local repository. If it's not there, it will automatically fetch it from the central repository.

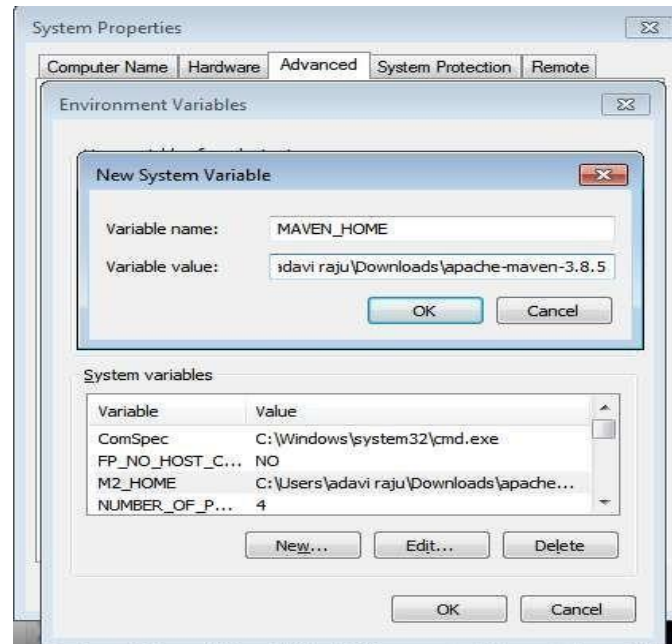
## 8. Installing and Setting Up Maven

To start using Maven, you need to install it and set it up on your computer.

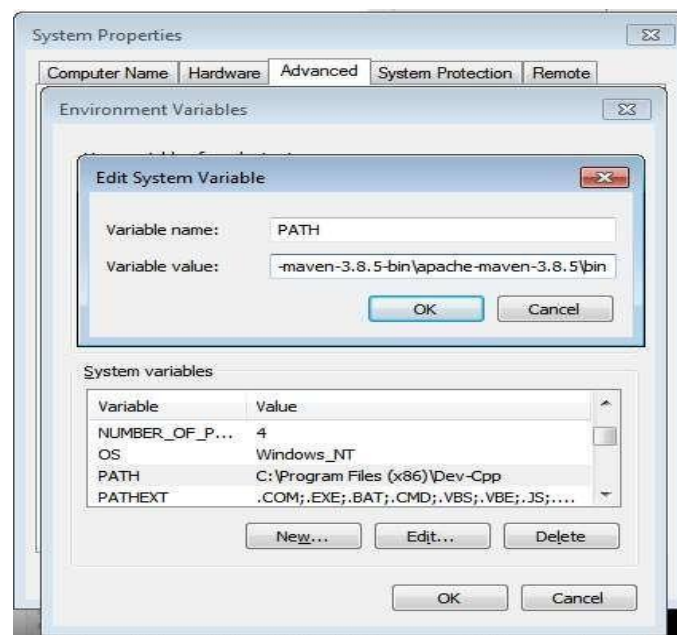
1. **Download Maven:** Get the latest version from the Apache website.
2. **Extract Files:** Unzip the downloaded files.



3. **Set Environment Variables:** You need to set up JAVA\_HOME (where Java is installed) and MAVEN\_HOME (where Maven is located) in your system settings.



4. **Add Maven to PATH:** This allows you to use Maven commands from any folder on your computer.



5. **Verification:** You can check if Maven is installed correctly by typing `mvn -version` in your terminal. If everything is set up right, it will show the version of Maven.